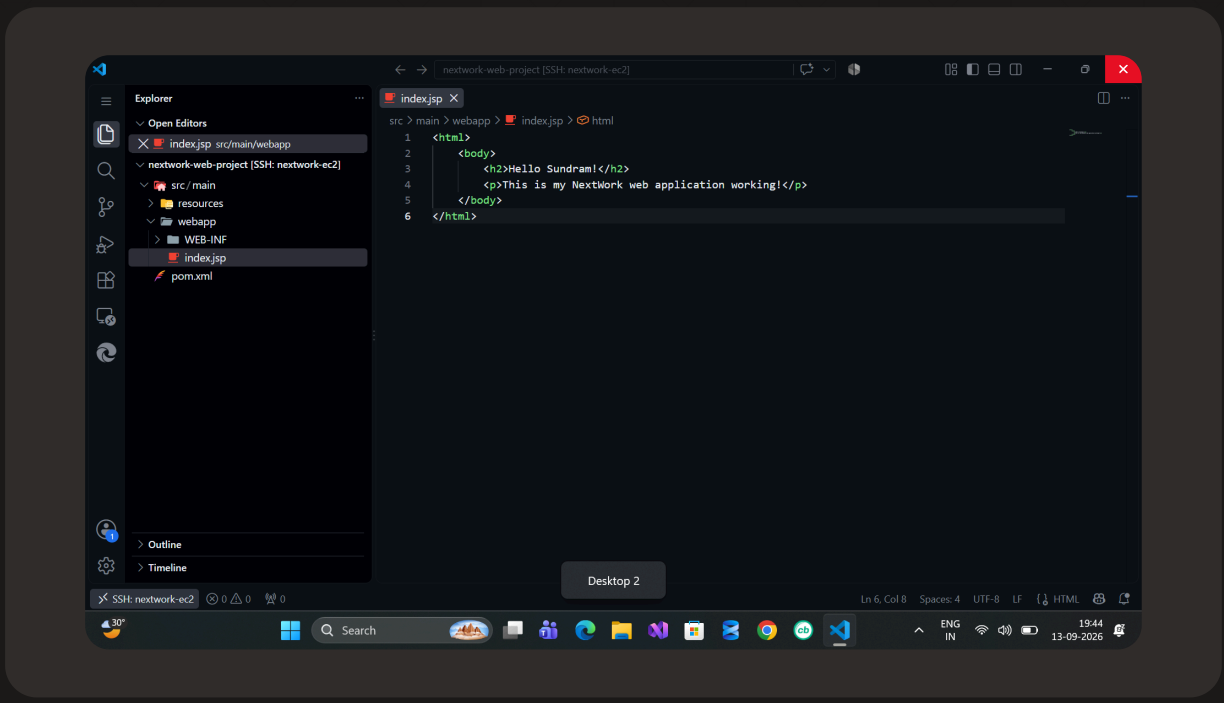
# SET UP A WEB APP IN THE CLOUD

By Sundram Singhal | September 2026



## Introducing Today's Project!

In this project, I will demonstrate... I'm doing this project to learn basics of building a AWS app along with Vs Code

This project is part one of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project from tomorrow itself.

I did this project because i wanna learn DEVOPS. and wanna add one more skill in my reusme.

## Key tools and concepts

Services I used were... Key concepts I learnt include creating my ec2 instance and then connectinf that instance with my vs code and vs code terminal.

## Project reflection

This project took me approximately... The most challenging part was connceting my instance with the vs code as it was disconnecting again and again. it cost me 4 hours but later on i solved the problem.

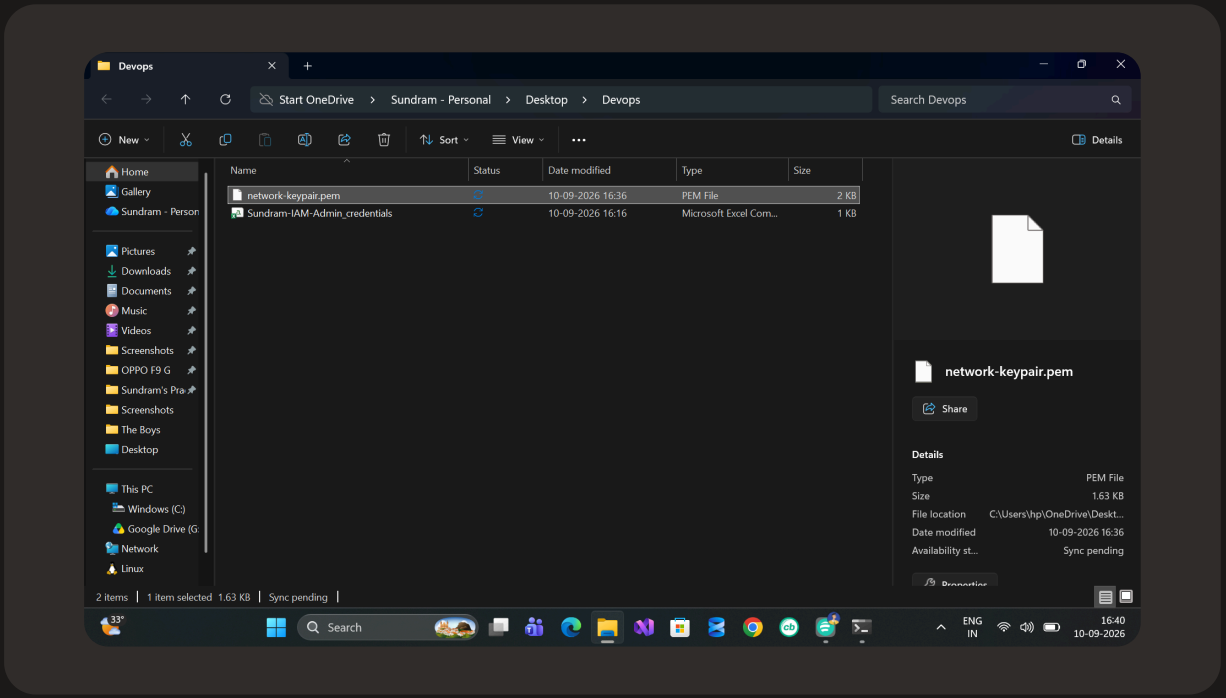
One thing I didn't expect in this project was connecting with vs code also it cost me 4 hours to connect as there was RAM problem which i solved by reading documentation itself.

# Launching an EC2 instance

## What I did in this step

In this step, I will Launch a new EC2 instance, Set up a key pair for secure access and set up network settings for my instance.

I started this project by launching an EC2 instance because i want to use AWS service which will allow me to rent a server instead of aquiring it on my local computer physically




## I also enabled SSH

SSH is... I enabled SSH so that only authorized user can access a remote server. When i will connect to my server later it will verify that i have the correct private key that matches the public key on the server.

## Key pairs

A key pair in EC2 is like the keys to your virtual computer, It lets you securely access your EC2 instance.

### Downloaded key pair file

Once I set up my key pair, AWS automatically downloaded .pem file which contains my private key

---

## Set up VS Code

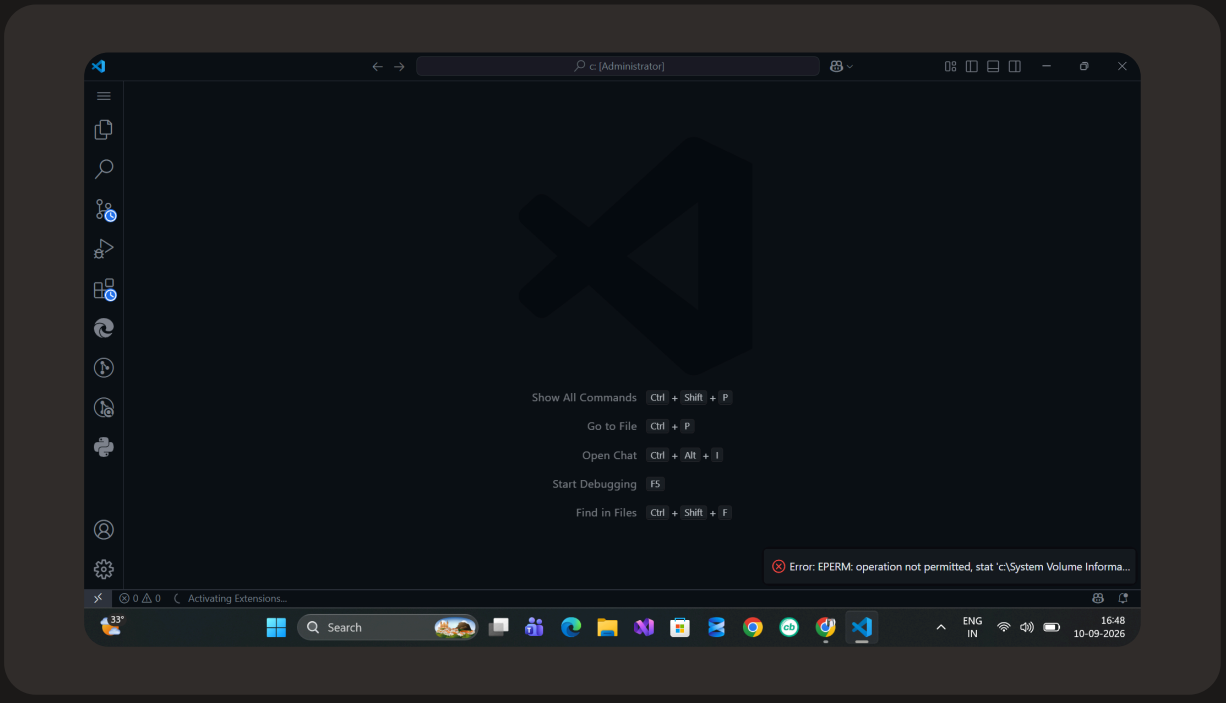
### What I did in this step

In this step, I will installl vs code because i want to connect with the instance i have created.

### What is VS Code?

VS Code is an IDE where i will connect with my created instance.

I installed VS Code to connect with my instance and create my first web app.

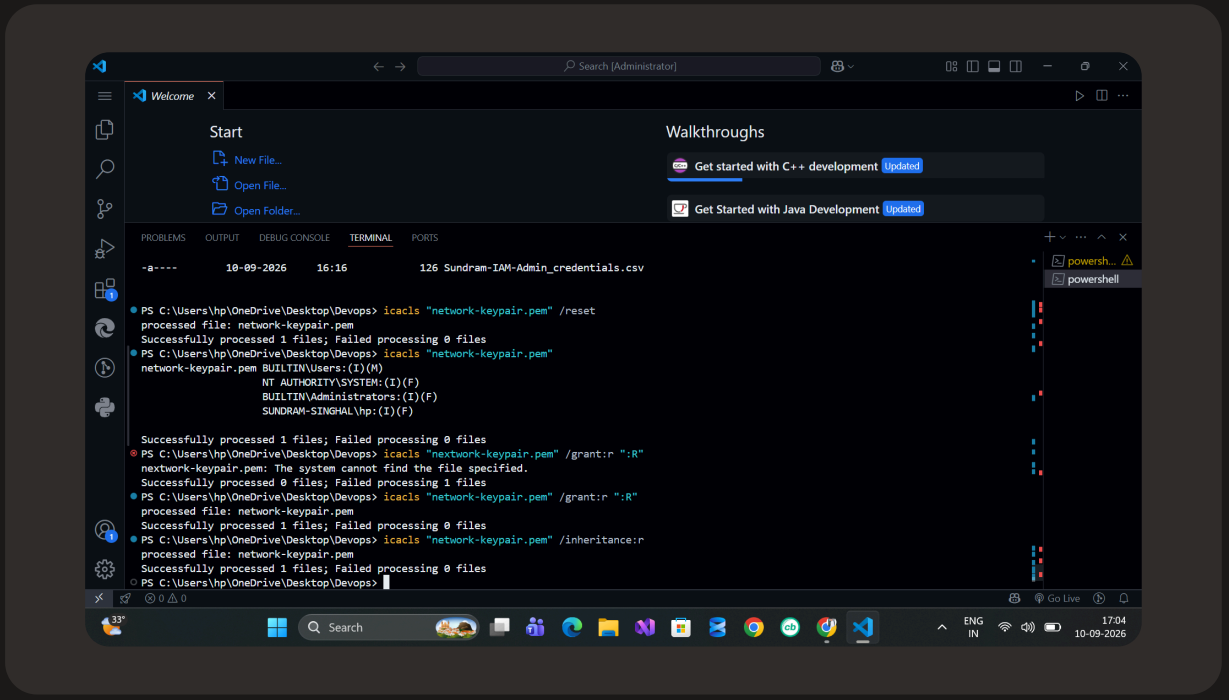


# My first terminal commands

A terminal is a way to talk to our computer by instructions instead of clicks. The first command I ran for this project is cd OneDrive

## Updating file permissions

I also updated my private key's permissions by using 3 commands and to ensure only i can access that.



# SSH connection to EC2 instance

## What I did in this step

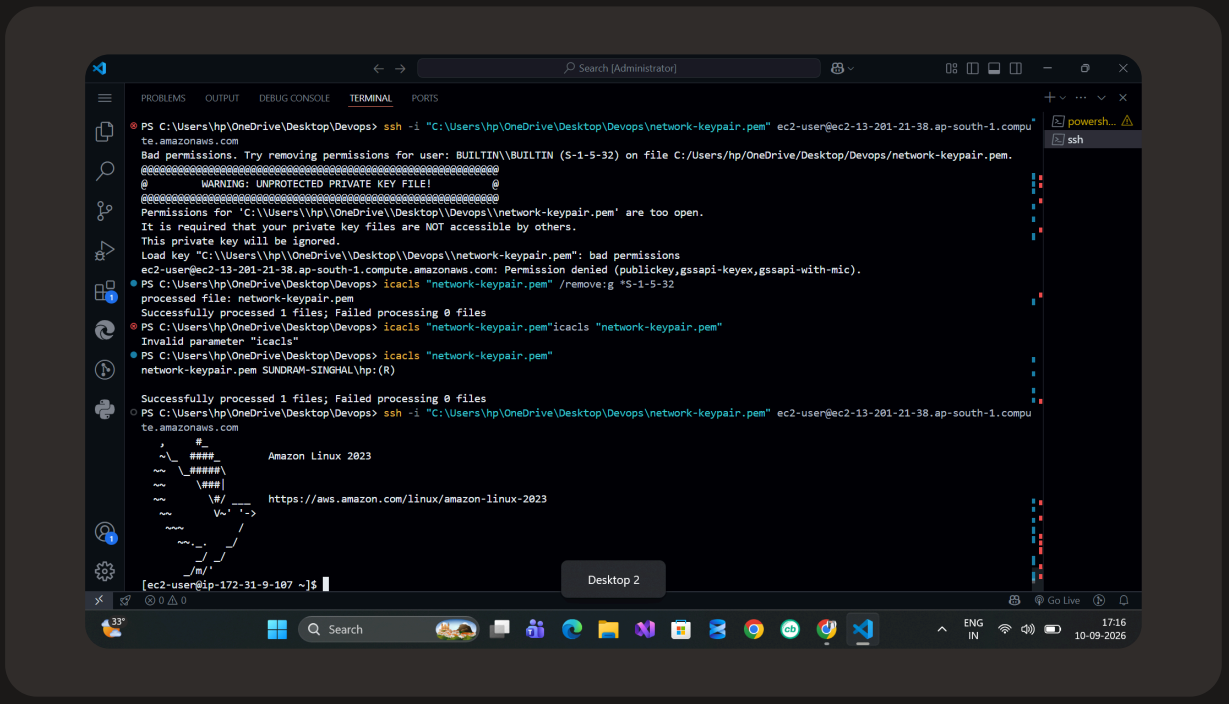
In this step, I will connect to our EC2 instance because i want to work inside my EC2 instance to set up that web app.

## Connecting to EC2

To connect to my EC2 instance, I ran the command listed above

## This command required an IPv4 address

A server's IPV4 DNS is the public address for my EC2 server that the internet uses to find and connect to it.



# Maven & Java

## What I did in this step

In this step, I will install apache maven on our EC2 instance, install amazon corretto 8, a version of java and verify the installations.

## Why I'm using Maven

Apache Maven is a tool that helps developers build and organize Java software projects. It's also a package manager, which means it automatically download any external pieces of code your project depends on to work.

Maven is required in this project because the application is a Java application, and Maven helps us build, manage dependencies, and package the application.

## Why I'm using Java

Java is a programming langauge that allows us to build different applications. Maven required java to build

Java is required in this project because maven required it to operate.

# Create the Application

## What I did in this step

In this step, I will run maven commands in my terminal because i want to generate a Java web app.

## Creating the Java web app

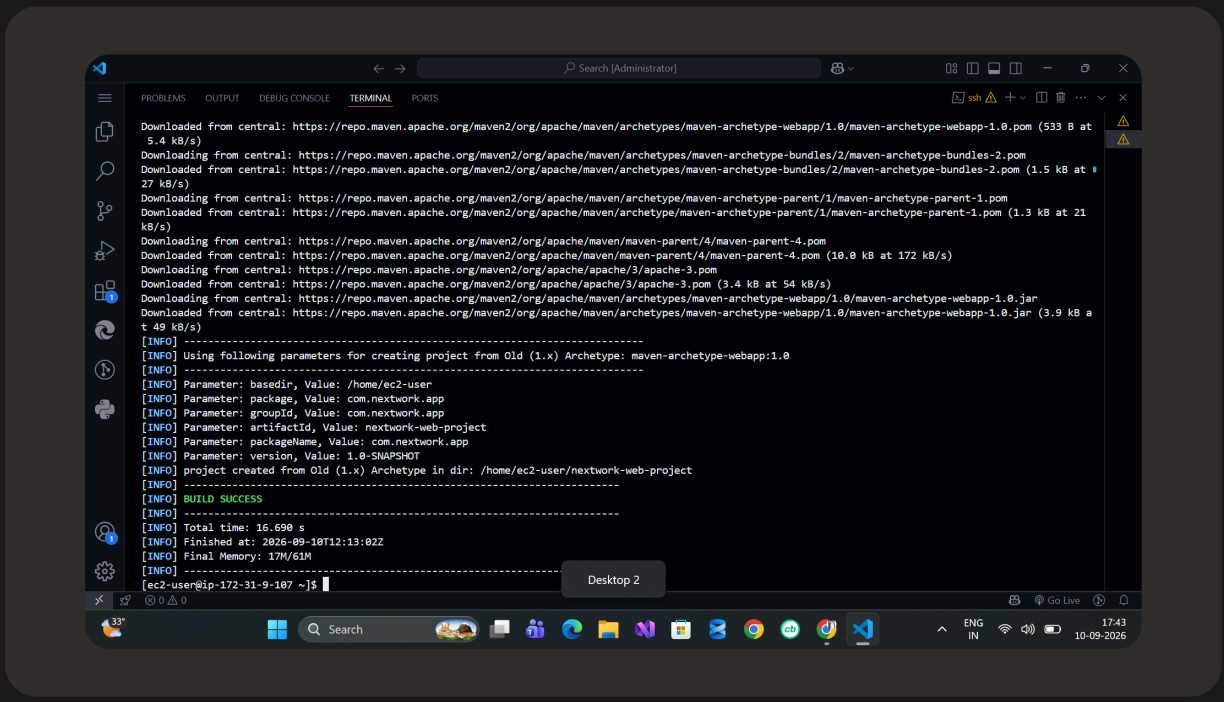
I generated a Java web app using the command listed above.

## Installing Remote - SSH

I installed Remote - SSH, which is it lets you connect directly via SSH to another computer securely over the internet. I installed it to vs code using vs code extensions.

## SSH configuration details

Configuration details required to set up a remote connection include HostName, IdentityFile and the User.

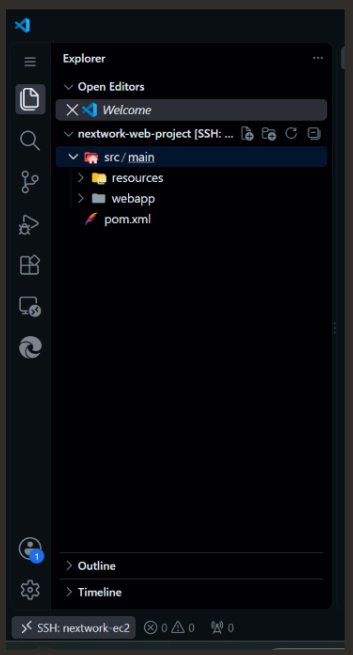


# Create the Application

## Exploring the project structure

Using VS Code's file explorer, I could see a folder in which there is a src folder which containes resource and webapp and a pom.xml file to handle and add the dependencies.

Two of the project folders created by Maven are src and webapp,  which are the web app's files e.g. HTML, CSS, JavaScript, and JSP files, and resources, which are the configuration files a web app might need e.g. connection settings to a database.



# Using Remote - SSH

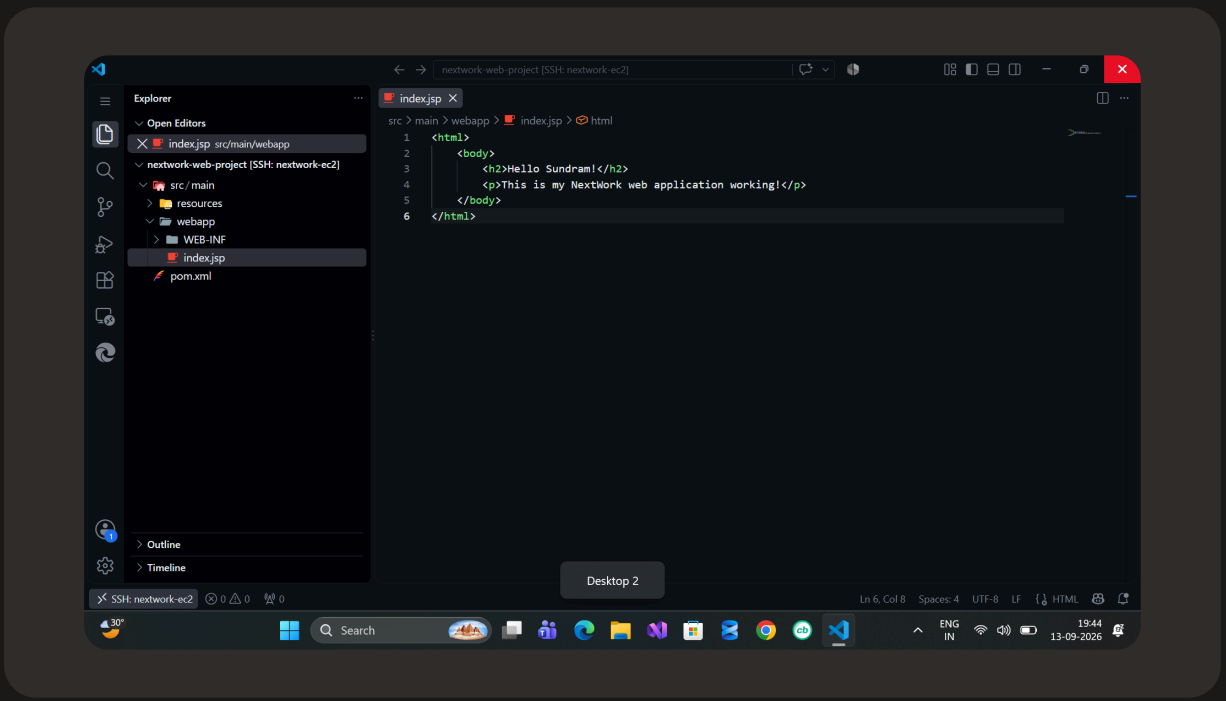
## What I did in this step

In this step, I will download an extension of vs code to connect EC2 directly with the IDE.

## Updating the web app

The index.jsp is a markup which we can use to make dynamic webpages and use java code inside that.

I edited index.jsp by adding h2 tag and paragraph tag and then save the updated file.



# Using nano

## Additional improvements

In this secret mission, I will change my .jsp file using the terminal

## Verifying my work

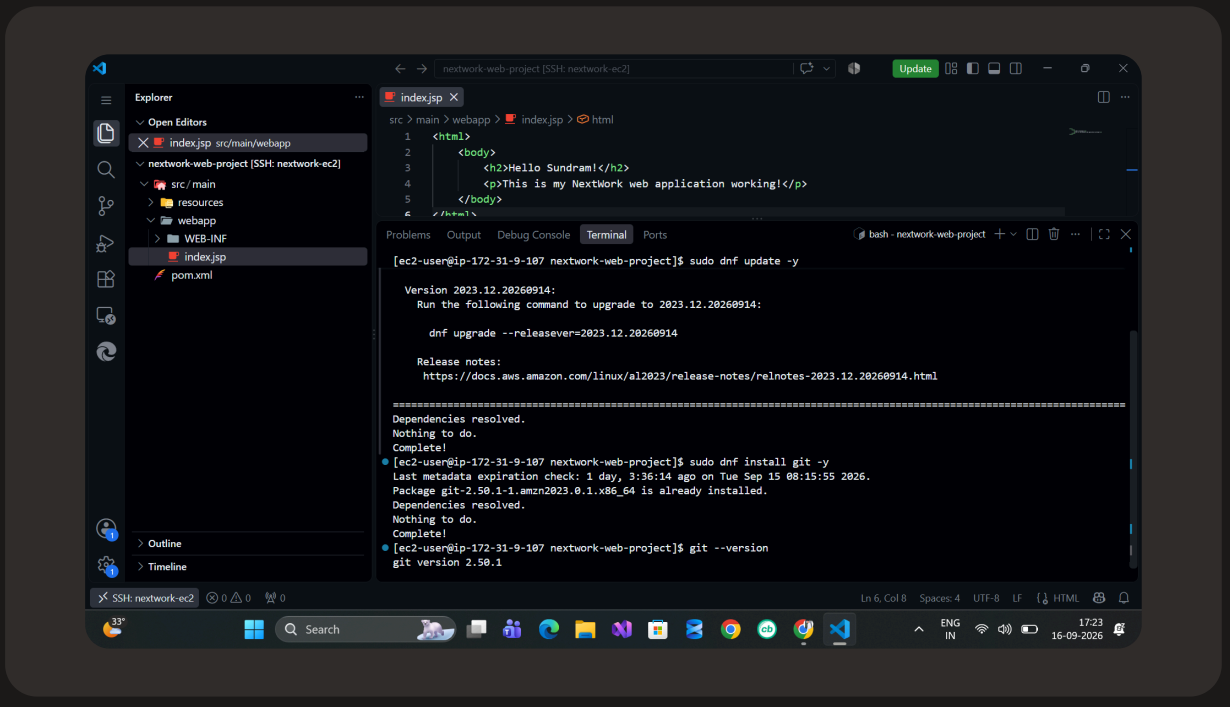
To verify my editing work in the terminal, I added a line of code using terminal. It was possible to see my changes in VS Code right away because i added a line of code in .jsp file

# CONNECT A GITHUB REPO WITH AWS

By Sundram Singhal | September 2026

---

## Connect a GitHub Repo with AWS



# Introducing Today's Project!

In this project, I will demonstrate git and github to track changes in our code. I'm doing this project to learn how ci/cd works.

## Key tools and concepts

Services I used were git and github mainly. Key concepts I learnt include how to connect our vs code with the github, how to push your code to github.

## Project reflection

This project took me approximately 1 hour 40 mins. The most challenging part was connceting with the instance again as i was continuing the course after 3 days. It was most rewarding to understand how we push our code to github and how to generate token and learn all the important commands.

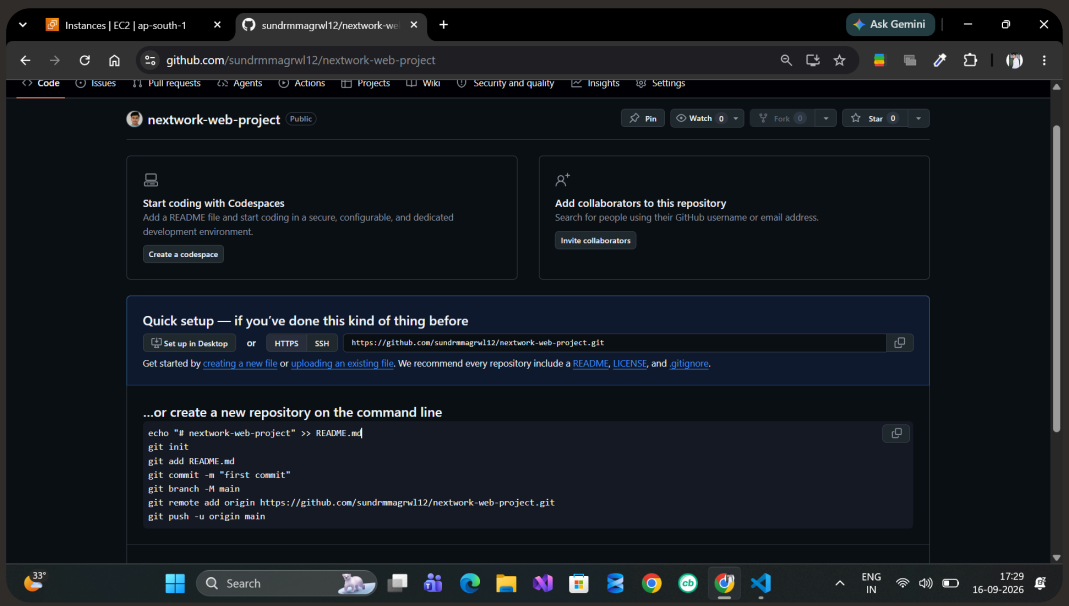
I did this project because i wanna complete this 6 days series as soon as possible and wanna be a guy who has good amount of DevOps AWS knowledge.

This project is part two of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tomorrow itself.

# Git and GitHub

Git is a version control to track changes in the code. I installed Git using the commands sudo dnf install git -y

GitHub is. like a cloud to store our code. I'm using GitHub in this project because it is user friendly and we can share our progess to anyone and can share our code.

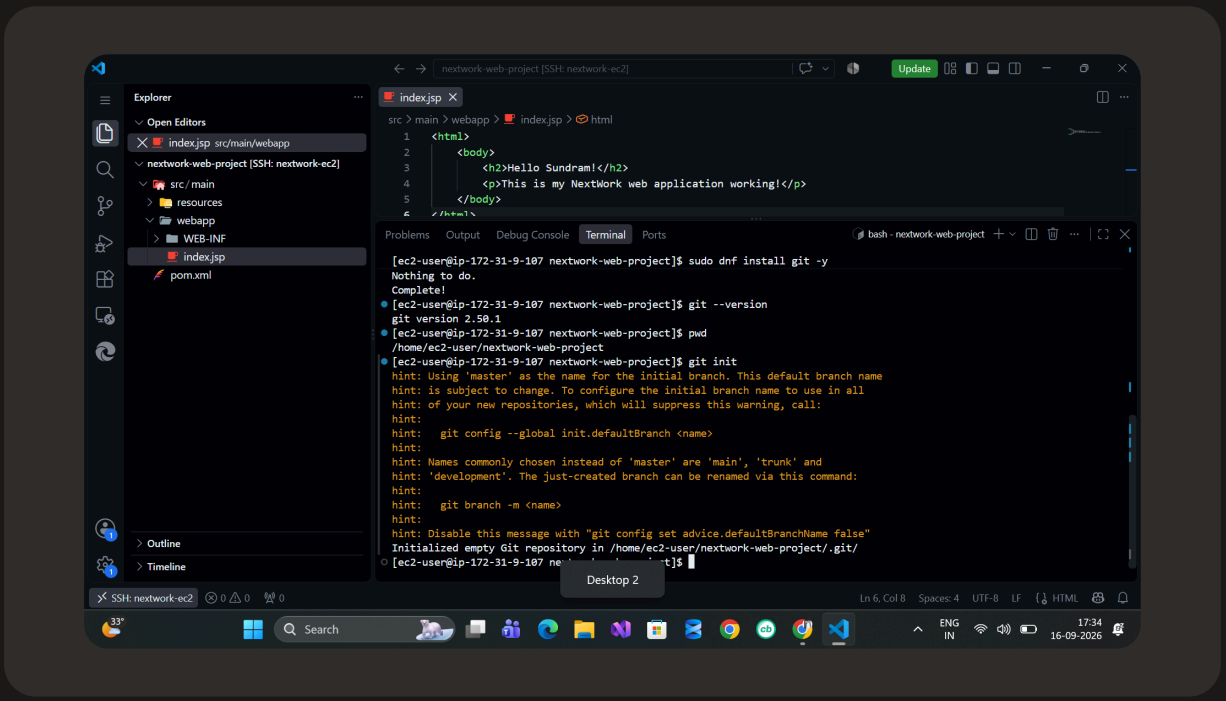


# My local repository

A Git repository is like folders that contain all your project files and their entire version history.

Git init is a command create a local repository on our computer. I ran this command in the vs code terminal.

A branch in Git is one of the universe of our code, means there can be multiple branches of the code. After running git init, the response from the terminal was a bunch of yellow messages indicating that i can set up my branch name.



# To push local changes to GitHub, I ran three commands

## git add

The first command I ran was git add . . A staging area is the set of all the files in which i have changed.

## git commit

The second command I ran was git commit -m "Updated index.jsp with new content". Using '-m' means i can send a message with my new commit indicating what changes i have made in this particular commit.

## git push

The third command I ran was git push -u origin master. Using '-u' means we're also setting an 'upstream' for your local branch, which means you're telling Git to remember to push to master by default.
Next time we can simply run git push

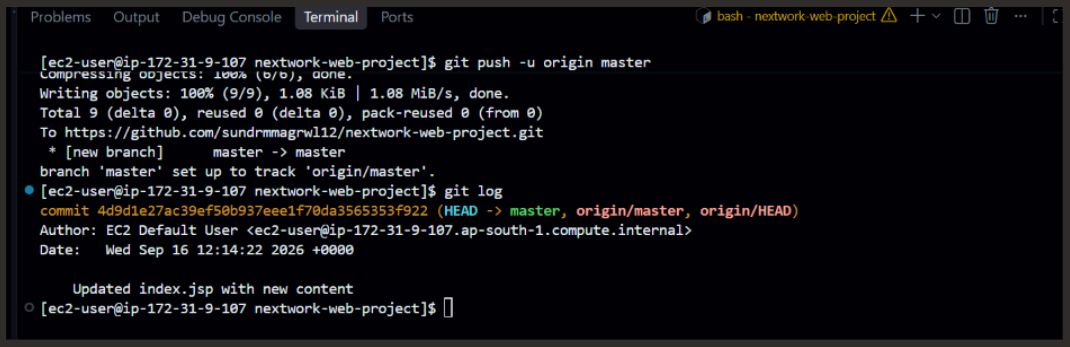
# Authentication

When I commit changes to GitHub, Git asks for my credentials because he wants to verify that only a verified user can change in repos.

## Local Git identity

Git needs my name and email because to keep track of the commits to see who made what change.

Running git log showed me the history of my commits including author, date with time.



# GitHub tokens

GitHub authentication failed when I entered my password because it expects a token instead of password because sending password can be risksy and passwords can get intercepted over the internet.

A GitHub token is like a key to provide vs code terminal to authenticate yourself. I'm using one in this project because i wanna connect my local repo with the GitHub repo.

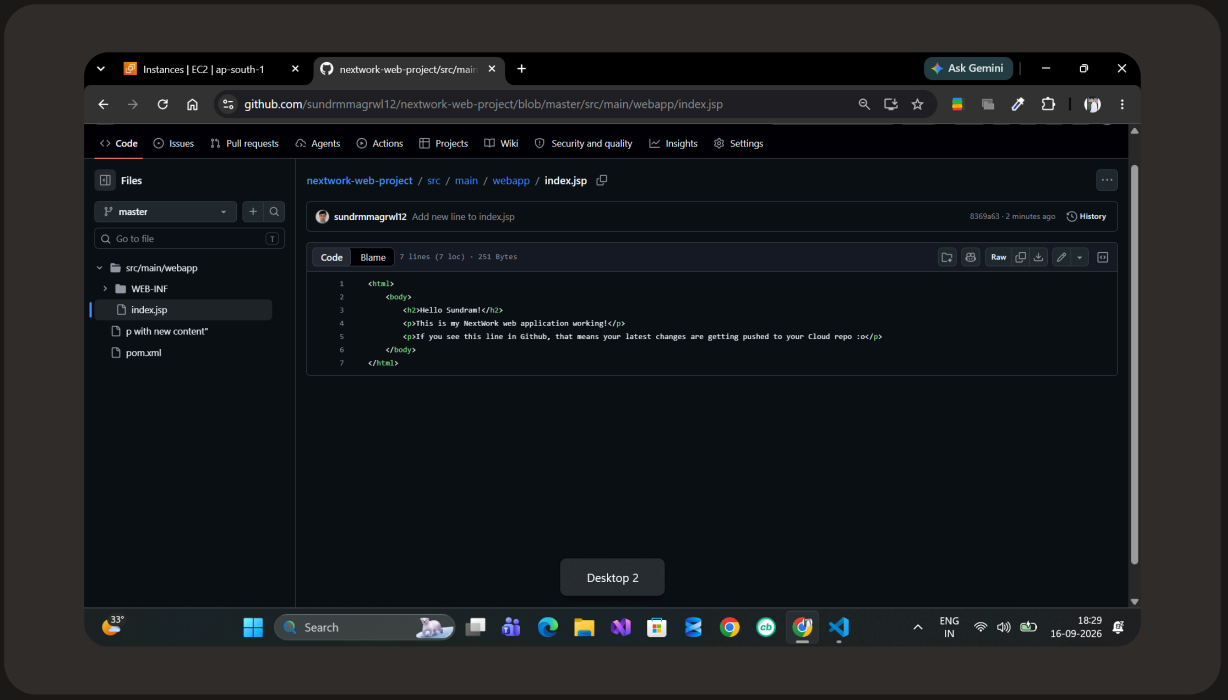
I could set up a GitHub token by going inside developer setting and choose the required stuffs and generate a token.
and then copy that token and save it for the future use to conncet our github repo with the local repo.



# Making changes again

I wanted to see Git working in action, so I edited my index.jsp. I couldn't see the changes in my GitHub repo initially because i did not commit those changes to the github they were only saved in my local repo not on the github repo.

I finally saw the changes in my GitHub repo after commiting those changes to my github repo by running this command - " git commit -m "Add new line to index.jsp" git push " which includes a message for the commit and then i enter my username and token to get authenticated.

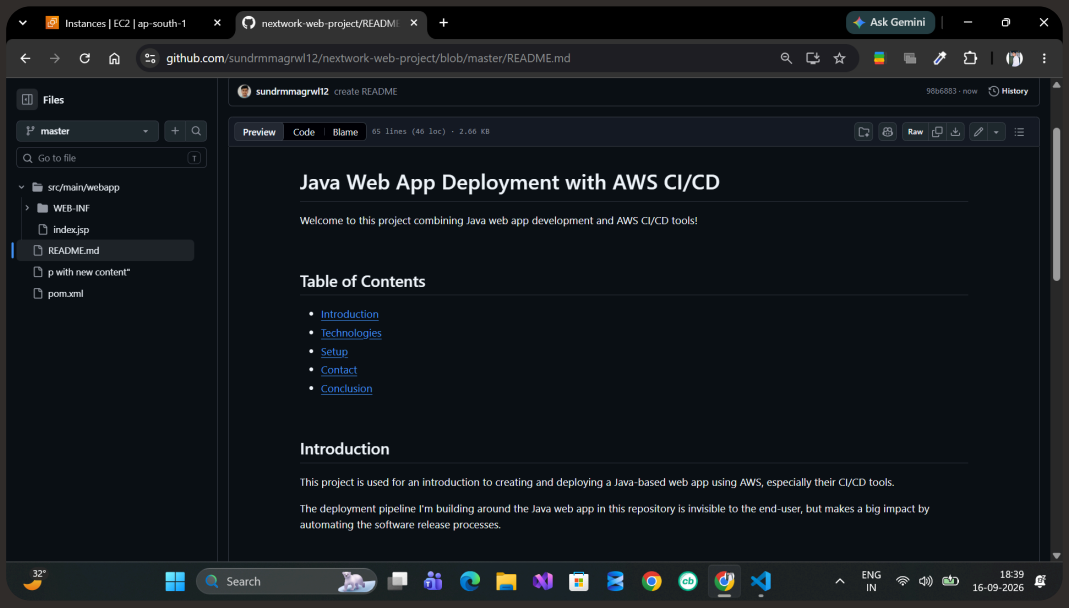


# Setting up a READMe file

As a finishing touch to my GitHub repository, I added a README file, which is the file with all the details about ,y project. I added a README file by using linux touch cmnd to create a new file.

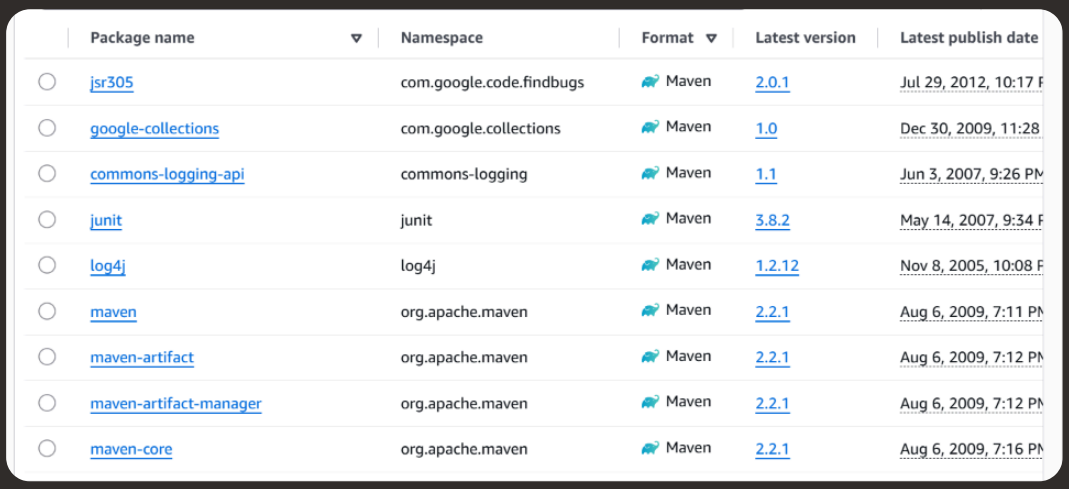
My README is written in Markdown because it is a file with Special characters can help you format text in Markdown, such as #, ##, ###, **, * etc.
this file helps others to understand our proejct properly and also instruct them how can they clone our project.

My README file has 6 sections that outline Introduction, Technologies, Setup, Contact mainly.



# SECURE PACKAGES WITH CODEARTIFACT

By Sundram Singhal | September 2026



| Package name | Namespace | Format | Latest version | Latest publish date |
|---|---|---|---|---|
| jsr305 | com.google.code.findbugs | Maven | 2.0.1 | Jul 29, 2012, 10:17 F |
| google-collections | com.google.collections | Maven | 1.0 | Dec 30, 2009, 11:28 |
| commons-logging-api | commons-logging | Maven | 1.1 | Jun 3, 2007, 9:26 PM |
| junit | junit | Maven | 3.8.2 | May 14, 2007, 9:34 F |
| log4j | log4j | Maven | 1.2.12 | Nov 8, 2005, 10:08 F |
| maven | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:11 PM |
| maven-artifact | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:12 PM |
| maven-artifact-manager | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:12 PM |
| maven-core | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:16 PM |

# Introducing Today's Project!

In this project, I will build CI/CD pipelines from scratch that will automate my building and deployment. I'm doing this project to learn how AWS CodeArtifact works.

## Key tools and concepts

Services I used were CodeArtifact, maven.Key concepts I learnt include is creating repository, set up the IAM role and then download all the required dependenices into our CodeArtifact.

## Project reflection

This project took me approximately 2 hours... The most challenging part was download the dependencies at the later part. It was most rewarding to finally complete the session.

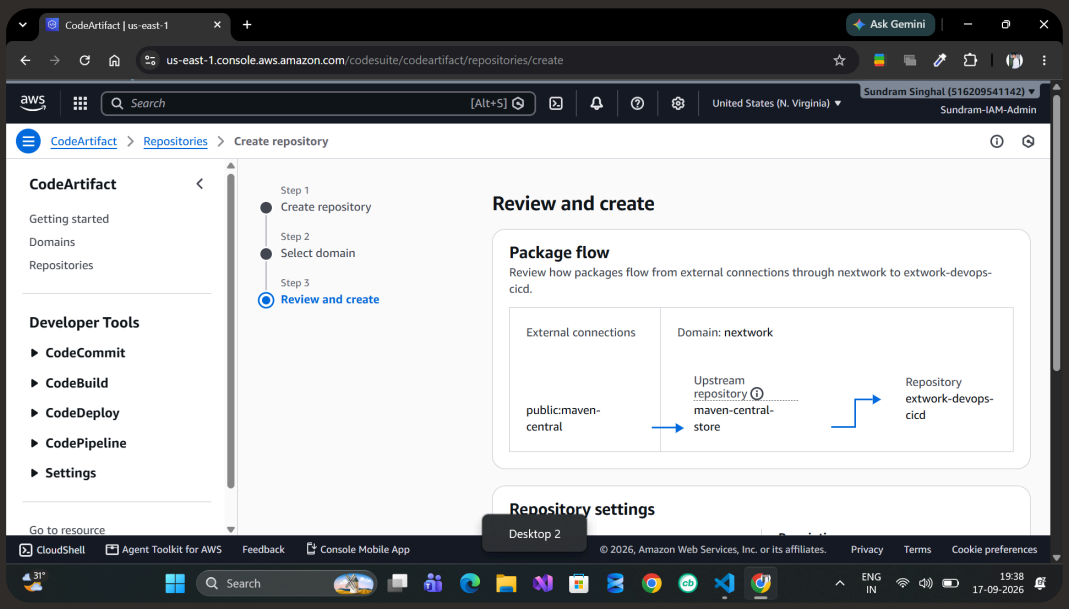
This project is part three of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tomorrow itself.

# CodeArtifact Repository

CodeArtifact is secure, central place to store all your software packages. Engineering teams use artifact repositories because it gives us a consistent, reliable place to store and retrieve these components. his gives you three big benefits: Security, Reliability and Control.

A domain is like a folder that holds multiple repositories belonging to the same project or organization.  My domain is CodeArtifact domain.

A CodeArtifact repository can have an upstream repository, which means our backup repository which can be useful when we require packages that are not available in the primary repository. My repository's upstream repository is maven central which contains all the important libraries and also all the java developers push their libraries here.



# CodeArtifact Security

## Issue

To access CodeArtifact, we need to run the command in Step 3 to give your terminal a temporary password.I ran into an error when retrieving a token because it says "I don't know who you are, so I can't let you access CodeArtifact."

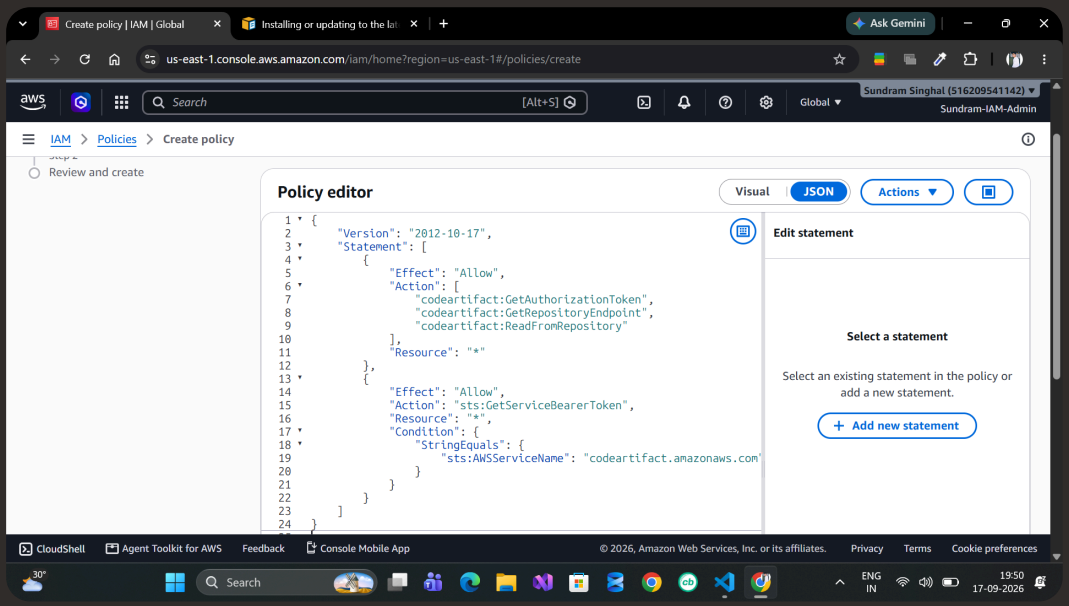
## Resolution

To resolve the error with my security token, beasically i was restarting after 24 hours so i created a new token and then paste in vs code and i did not get any error.

It's security best practice to use IAM roles because IAM roles are a security best practice because they provide temporary credentials and allow applications to access AWS resources without storing long-term credentials, while supporting least-privilege access.

# The JSON policy attached to my role

The JSON policy I set up grants (codeartifact:* actions) gives permission to get authentication tokens, find repository locations, and read packages from repositories.
The second part (sts:GetServiceBearerToken) allows temporarily elevated access specifically for CodeArtifact operations.

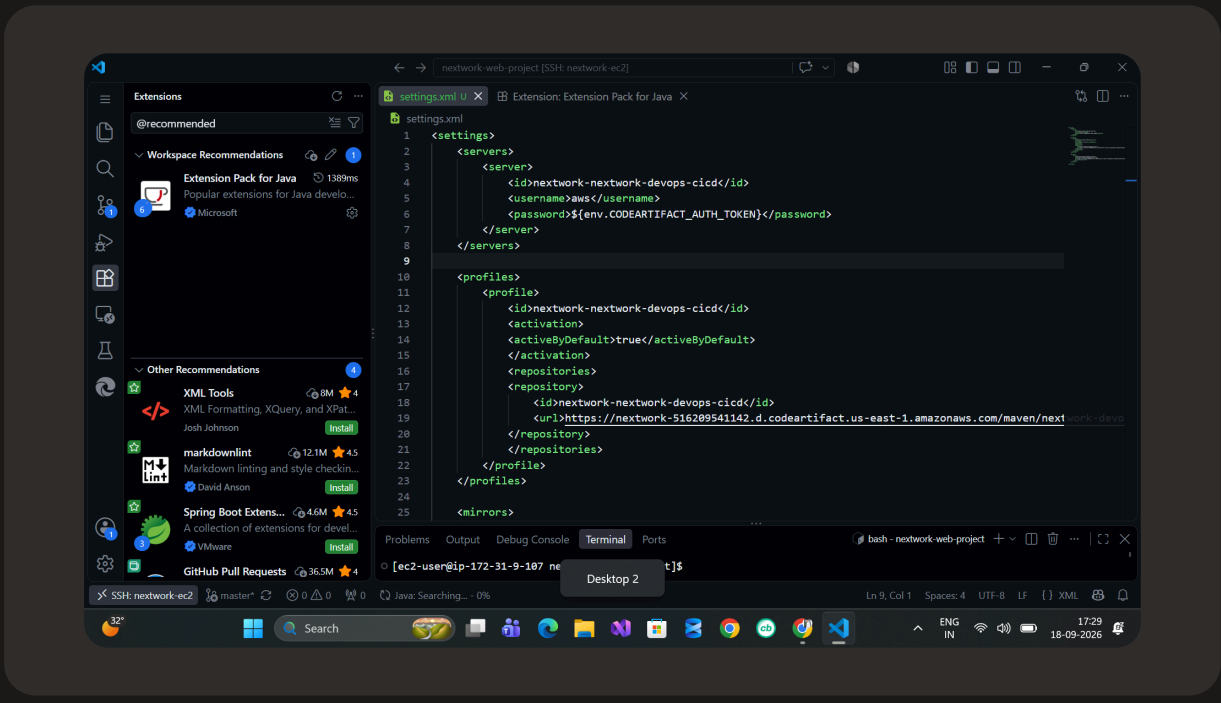


# Maven and CodeArtifact

## To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

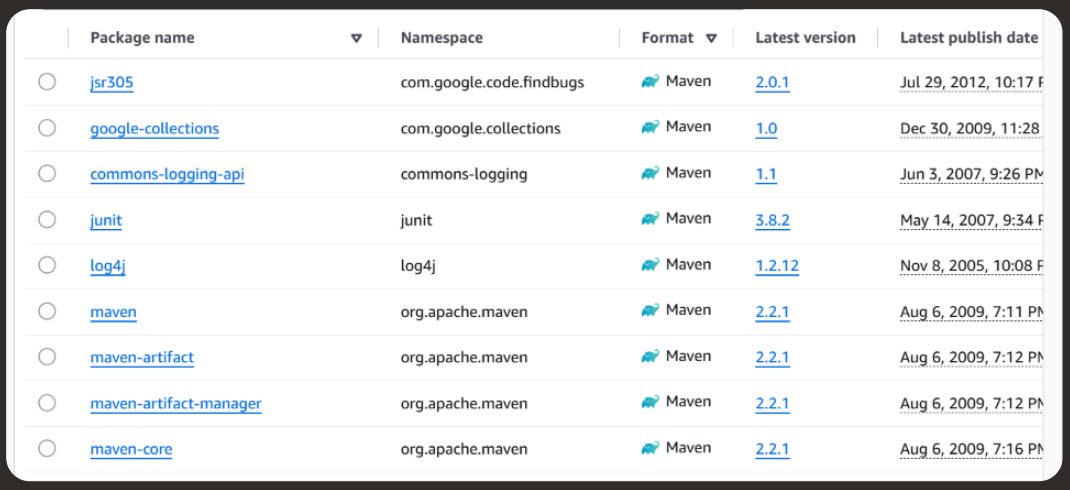
The settings.xml file configures Maven to work with codeartifact because we gave it the required information to authentication maven like token and rest of all the required stuffs.

Compiling means translating your project's code into a language that computers can understand and run. When you compile your project, you're making sure everything is correctly set up and ready to turn into a working app.



# Verify Connection

After compiling, I checked nextwork-devops-cicd Repository. I noticed a list of so many packages there.



| Package name | Namespace | Format | Latest version | Latest publish date |
|---|---|---|---|---|
| jsr305 | com.google.code.findbugs | Maven | 2.0.1 | Jul 29, 2012, 10:17 F |
| google-collections | com.google.collections | Maven | 1.0 | Dec 30, 2009, 11:28 |
| commons-logging-api | commons-logging | Maven | 1.1 | Jun 3, 2007, 9:26 PM |
| junit | junit | Maven | 3.8.2 | May 14, 2007, 9:34 F |
| log4j | log4j | Maven | 1.2.12 | Nov 8, 2005, 10:08 F |
| maven | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:11 PM |
| maven-artifact | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:12 PM |
| maven-artifact-manager | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:12 PM |
| maven-core | org.apache.maven | Maven | 2.2.1 | Aug 6, 2009, 7:16 PM |

## Uploading My Own Packages

In a project extension, I also decided to complete secret mission which is to create my Secret Package. This is useful in situations where i want to share my code internally but wanna keep that private from the outside world.

To create my own package, I... I also generated a security hash because...To create my own package, I open the CloudShell terminal and then ran the required commands. I also generated a security hash because it helps me to protect my package from others, As it provides security.

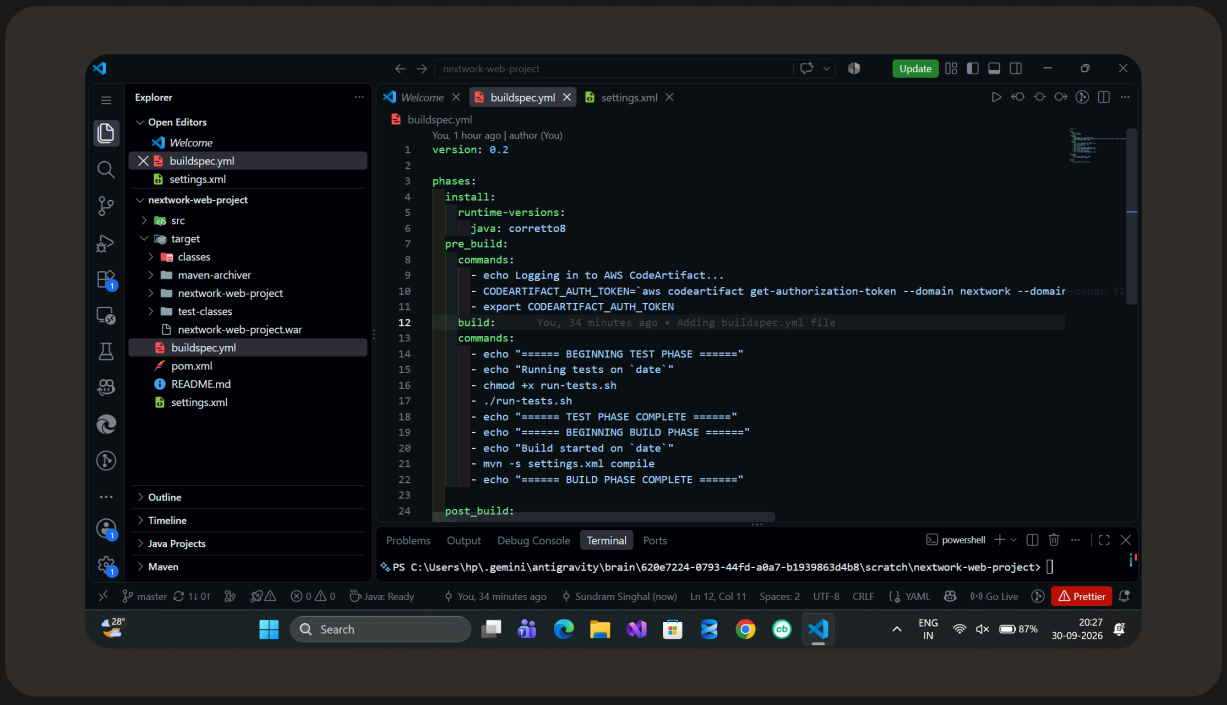
To publish the package,  When I look at the package details in CodeArtifact, I can see all the details about my newly added package there.

To validate my packages, I ran a command and then check the output to verify them.



# CONTINUOUS INTEGRATION WITH CODEBUILD

By Sundram Singhal | September 2026



# Introducing Today's Project!

In this project, I will demonstrate AWS CodeBuild to automate the build process for your web app. 'm doing this project to learn how continuous deployment works when the developer write the code and then how it get build tested and deployed.

## Key tools and concepts

Services I used were: AWS CodeBuild, AWS CodeArtifact, Amazon S3, AWS CodeConnections, Amazon CloudWatch Logs, AWS IAM, and GitHub. Key concepts I learnt include: Continuous Integration (CI) automation, writing buildspec.yml build pipelines, automated testing with test scripts, build artifact packaging (.war/.zip to S3), and managing IAM service role permissions.

## Project reflection

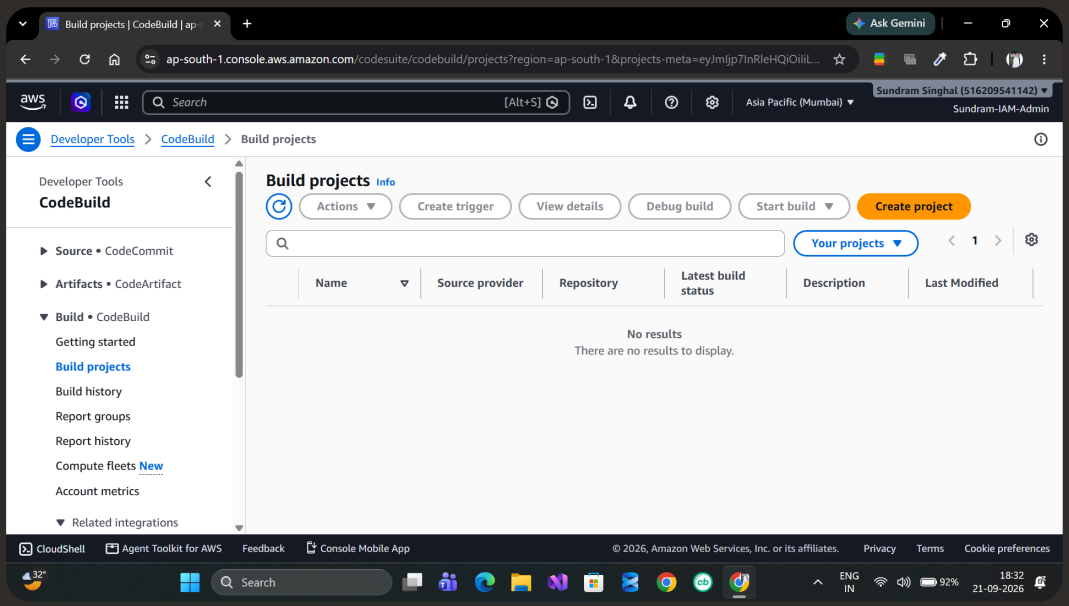
This project took me approximately 3 hours. The most challenging part was to start codebuild as there was a limit issue in my region. It was most rewarding to complete the tutorial.

This project is part four of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project by tomorrow itself.

# Setting up a CodeBuild Project

CodeBuild is a continuous integration service, which means the code developer write is automatically run, compile and tested. Engineering teams use it because it automates the tasks and reduce the manual works.

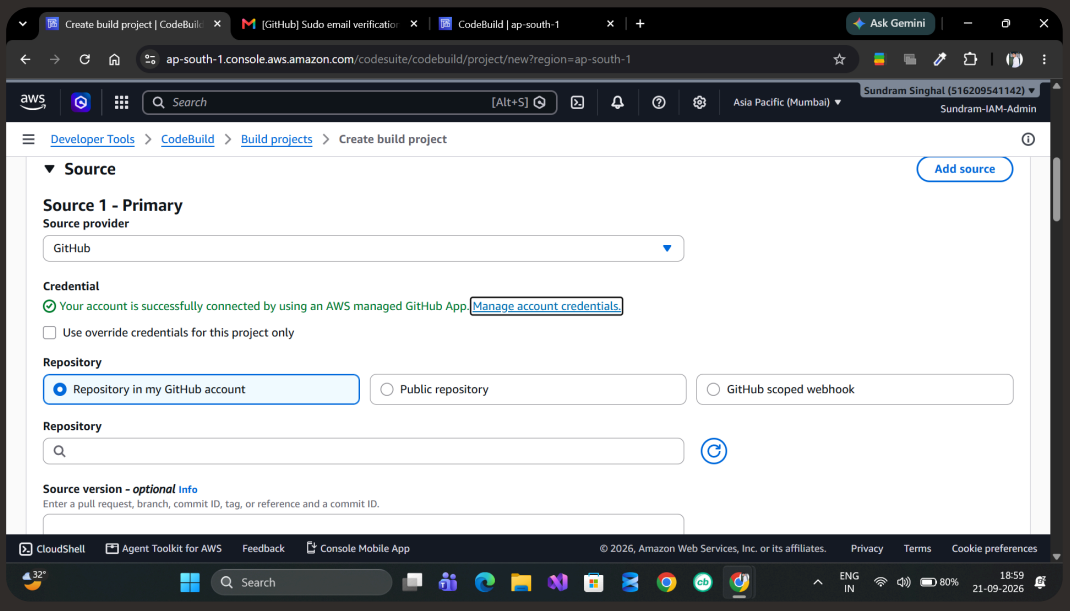
My CodeBuild project's source configuration means the location from where out CodeBuild will take the code form. and I selected GitHub because i will keep all my updated code on GitHub.



# Connecting CodeBuild with GitHub

There are multiple credential types for GitHub, like GitHub app, Personal access token and OAuth app. I used GitHub App because AWS manages the application and connection, reducing the need for you to handle tokens or keys directly. It's recommended for most use cases due to its ease of use and enhanced security.

The service that helped connect is CodeConnections. AWS CodeConnections is like a secure bridge between AWS and your external code repositories. Instead of dealing with the headache of managing API keys, tokens (like GitHub's Personal Access Tokens!), or SSH credentials, CodeConnections handles all that authentication complexity for you - so you can focus on building your application.



# CodeBuild Configurations

## Environment

My CodeBuild project's Environment configuration means the environment i need in my project. It includes settings like provisioning model, Environment image, Compute and Running Code.

## Artifacts

Build artifacts are the tangible outputs of our build process. They're what you'll actually deploy to your servers or distribute to users. That's why storing them, i created a S3 bucken to store  properly in S3 is so important - they're the whole reason we're running the build in the first place.

## Packaging

When setting up CodeBuild, I also chose to package artifacts in a zip file because Zip compression can significantly reduce the size of your artifacts, which means faster uploads to S3, Instead of managing multiple individual files, you get one tidy package and When it comes time to deploy your application or share it with teammates, having a single zip file makes the process much more straightforward.

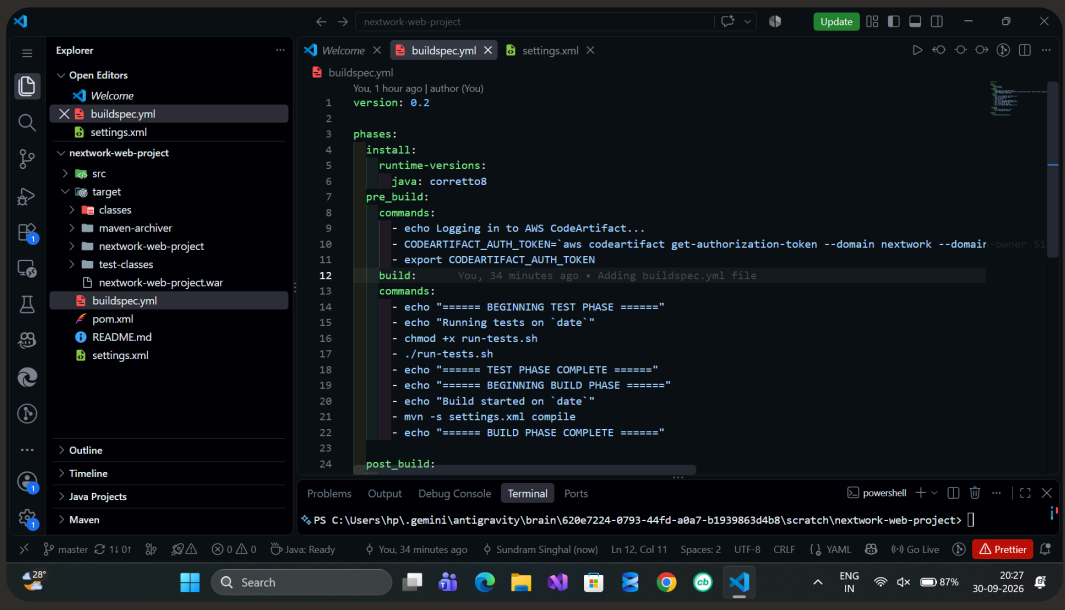
## Monitoring

For monitoring, I enabled CloudWatch Logs because it is a monitoring service that collects and tracks logs from AWS services. In this project, CloudWatch will record everything that happens during the build process, including the commands that are run, the output of those commands, and any errors that occur.

# buildspec.yml

My first build failed because I had not added the buildspec.yml file to my GitHub repository. A buildspec.yml file is needed because it provides all the required build instructions and phases that AWS CodeBuild searches for to compile, test, and package the application.

The first two phases in my buildspec.yml file... The third phase in my buildspec.yml file... The fourth phase in my buildspec.yml file..The first two phases in my buildspec.yml file (install and pre_build) set up the Java 8 runtime environment and log in to AWS CodeArtifact to fetch the authorization token.
The third phase in my buildspec.yml file (build) executes automated unit tests (run-tests.sh) and compiles the Java source code using Maven.
The fourth phase in my buildspec.yml file (post_build) packages the compiled application into a final .war web archive ready for deployment.

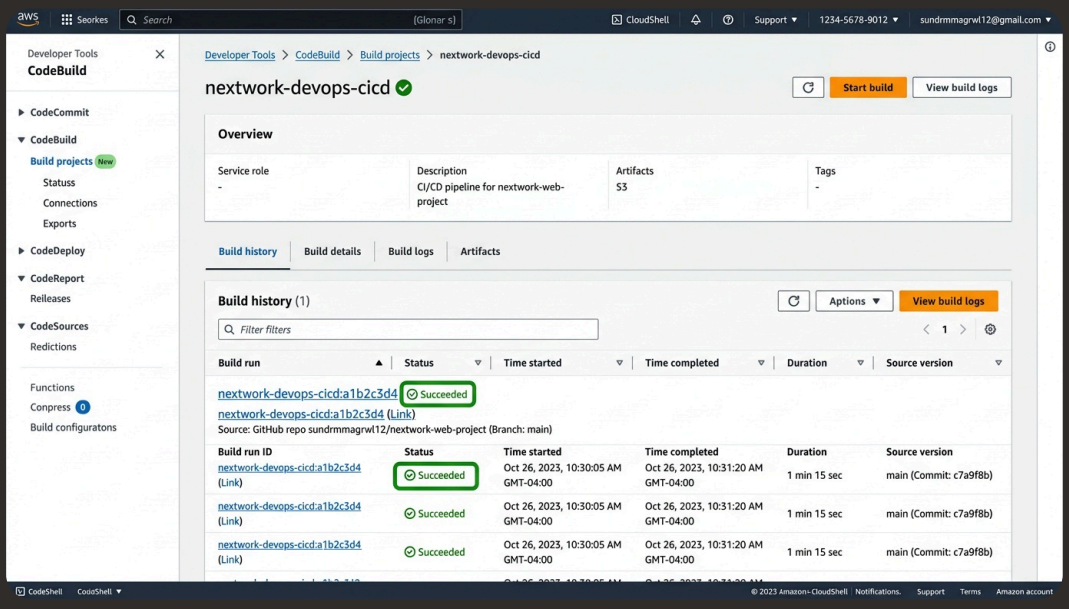


# Success!

My second build also failed, but with a different error that said  our CodeBuild service role doesn't have permission to access CodeArtifact, which is needed to download project dependencies.To fix this we need to grant CodeBuild's IAM role the permission to access CodeArtifact.

To resolve the second error, I attached the codeartifact-nextwork-consumer-policy IAM policy to the CodeBuild service role (codebuild-nextwork-devops-cicd-service-role). This gave CodeBuild the necessary permissions to authenticate with AWS CodeArtifact, retrieve the authorization token, and access project dependencies using settings.xml. When I built my project again, I saw the build status change to Succeeded with a green checkmark! All build phases (install, pre_build, build, and post_build) executed cleanly, running the automated test script and successfully packaging the .war web application artifact into our S3 bucket.

To verify the build, I checked my Amazon S3 bucket (nextwork-devops-cicd-sundram) in the AWS Console to ensure the build artifact was properly generated and saved. Seeing the artifact tells me that our web application code was successfully compiled, unit tests passed without errors, and the final .war bundle (nextwork-devops-cicd-artifact.zip) was packaged and securely stored in S3, ready for deployment in the next stage of our CI/CD pipeline.

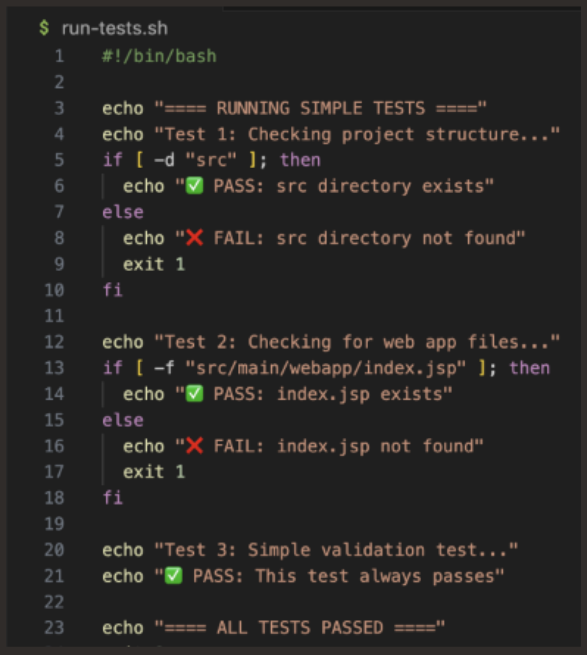


## Automating Testing

In a project extension, I'm adding a new file named run-tests.sh that performs a simple validation tests on your project structure - is there a src folder and an index.jsp file in your project directory?

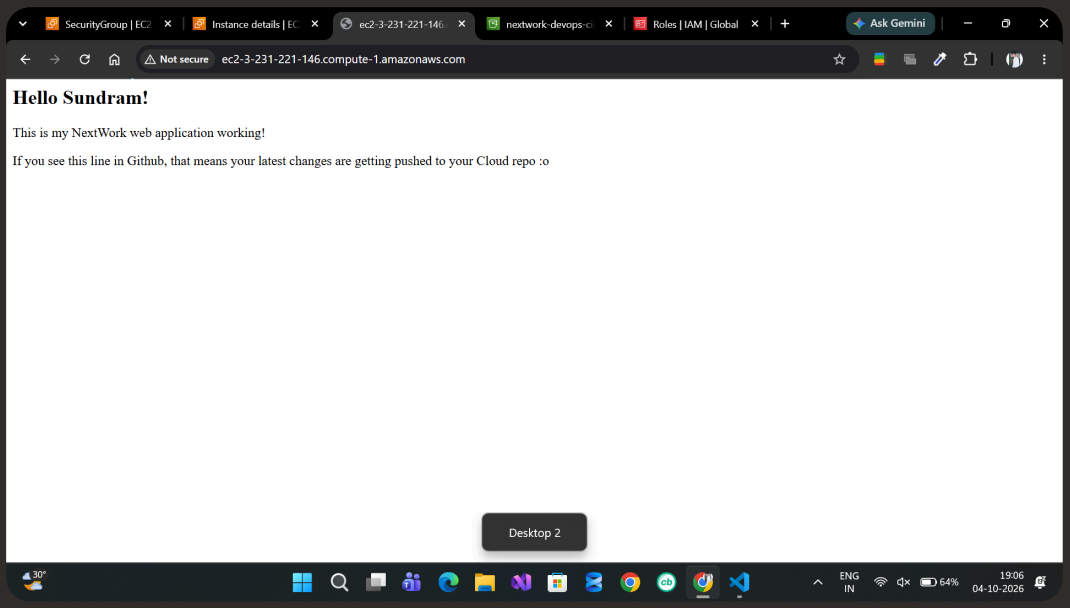
To add the test script to the build process, I created an automated shell script named run-tests.sh in the project root directory to validate our application's structure and key files (like src and index.jsp). Then, I updated the buildspec.yml file to include execution permissions (chmod +x run-tests.sh) and run the test script (./run-tests.sh) during the build phase before compiling and packaging the application.

After pushing my code to GitHub, I ran a new build in AWS CodeBuild by clicking Start build to trigger our updated CI pipeline with automated testing. I could see the build complete with a status of Succeeded, and checking the Build logs confirmed that CodeBuild automatically executed our run-tests.sh script, printed all passing test markers (PASS), and successfully compiled the final .war application package.



# DEPLOY A WEB APP WITH CODEDEPLOY

By Sundram Singhal | October 2026



# Introducing Today's Project!

In this project, I will demonstrate how to automate the deployment I'm doing this project to learn how automatic deployment works, how it handle the errors and rollback accordingly and allow us not to copy files.

## Key tools and concepts

Services I used were AWS EC2, AWS S3, AWS CodeArtifact, AWS CloudFormation, AWS CodeDeploy, AWS IAM, and GitHub. Key concepts I learnt include CI/CD pipeline automation, Infrastructure as Code (IaC) with CloudFormation templates, building & packaging Java web artifacts using Maven, managing CodeDeploy lifecycle events with appspec.yml, shell scripting for deployment automation, and scoping security with IAM roles and Security Groups.

## Project reflection

This project took me approximately 3 hours to complete. The most challenging part was setting up the codebuild part as in my region the limit was 0 then i build by project from the vs code terminal itself. It was most rewarding to complete the project and i feeled extremly satisfied when i saw that my project was deployed successfully.

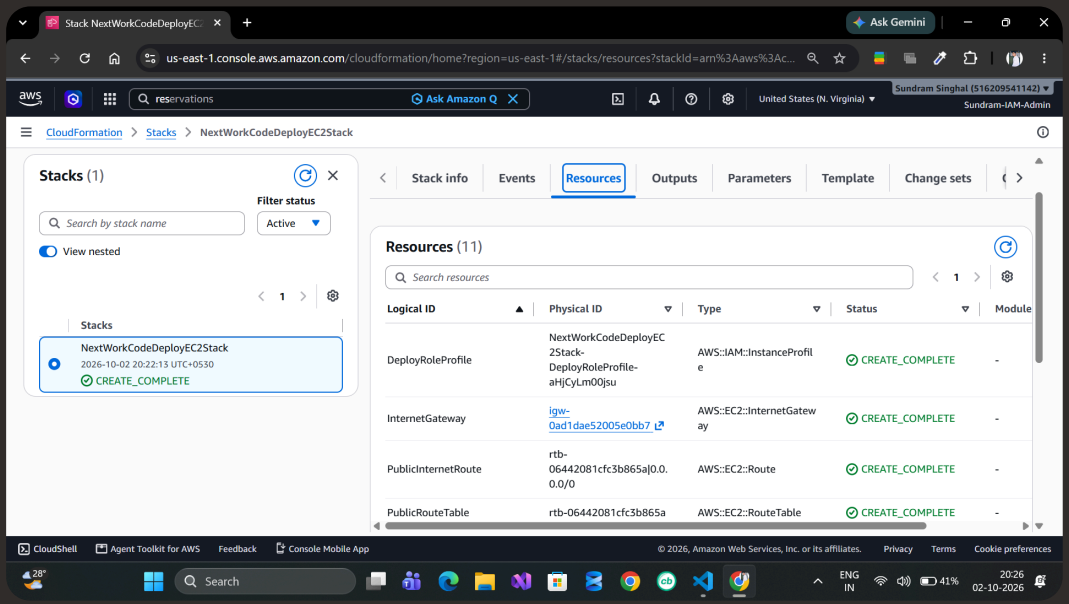
This project is part five of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project from tomorrow itself.

# Deployment Environment

To set up for CodeDeploy, I launched an EC2 instance and VPC because the EC2 instance acts as the web server to host and run our Java web application, while the VPC provides a secure, isolated network for it.

Instead of launching these resources manually, I used AWS CloudFormation to automatically create and configure all infrastructure using a template file.
When I need to delete these resources, I can simply delete the CloudFormation stack, which will automatically remove all created resources at once.

Other resources created in this template include Security Groups, Subnets, an Internet Gateway, and an IAM Instance Profile (Role). They're also in the template because they provide internet connectivity, set up network routing, define firewall security rules to allow traffic from my IP, and grant the EC2 instance permissions to talk to CodeDeploy and S3.



# Deployment Scripts

Scripts are like mini-programs that automate tasks. They're essentially text files filled with commands that you'd normally type one by one in the terminal, but packaged together so they run automatically in sequence. In our project, we're using scripts to automate deployment tasks.

The install_dependencies.sh script sets up all the software needed to run our website by installing programs (called Tomcat and Apache) that handle internet traffic and host our application. It then creates special settings that let these programs to work together, making our website accessible to visitors on the internet.

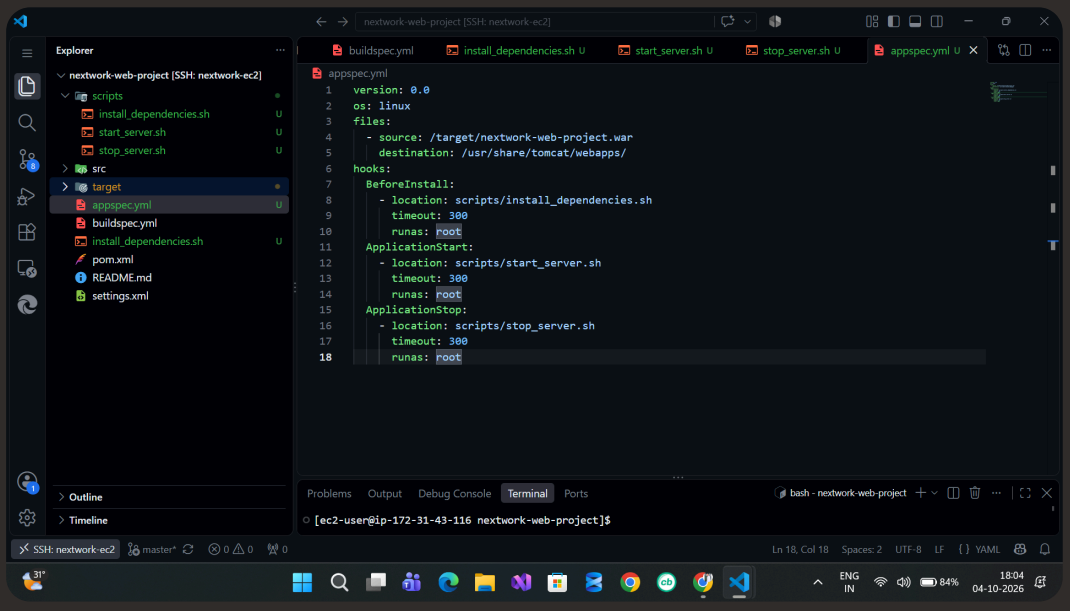
The start_server.sh starts both Tomcat (our Java application server) and Apache (our web server) and makes sure they'll restart automatically if the EC2 instance ever reboots.

The stop_server.sh will safely stops web server services by first checking if they're running. It uses pgrep to check for running processes of Apache (httpd) and Tomcat, and only attempts to stop the services if they are actually active. This prevents errors that could occur from trying to stop services that aren't running.

# appspec.yml

Then, I wrote an appspec.yml file to provide instruction manual for CodeDeploy. The key sections in appspec.yml are version, os, files, hooks.
We can assume appspec.yml file as the choreographer that ensures everyone moves in the right sequence during deployment.

I also updated buildspec.yml because we are telling CodeBuild: "After you build the app, make sure to include these additional files in the final package."  We're adding our appspec.yml and scripts folder because CodeDeploy will need them to properly deploy the application. Without them, CodeDeploy wouldn't know what to do with our compiled code!



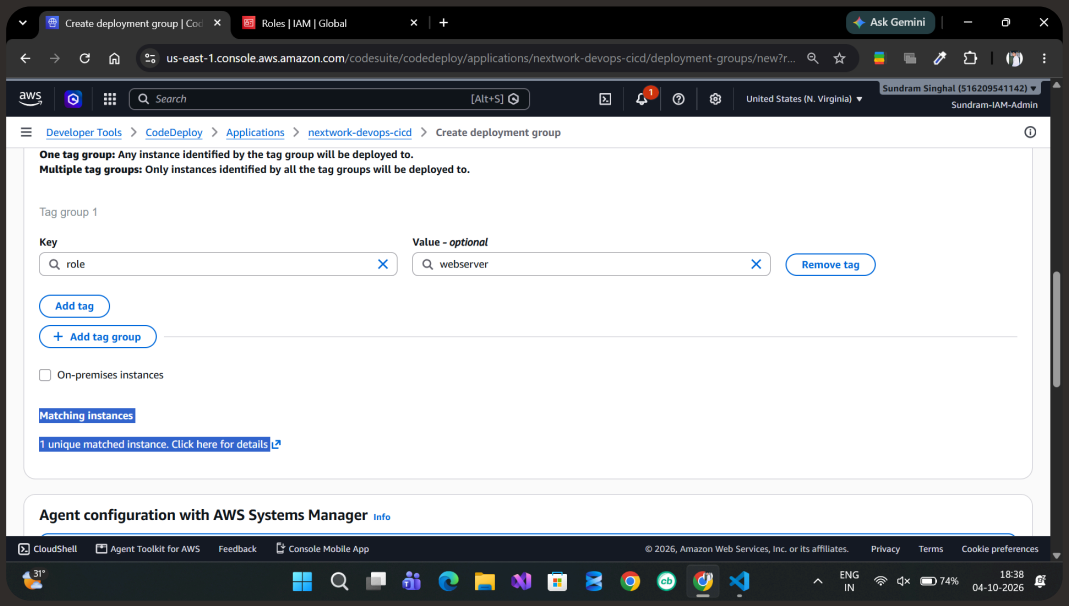
# Setting Up CodeDeploy

A deployment group is a collection of EC2 instances that are grouped to deploy something together and A CodeDeploy application is like the main folder for your deployment project. It doesn't do much on its own, but it helps you organize everything related to deploying one application.

CodeDeploy needs IAM roles to get permissions to access and manage AWS resources on your behalf. These permissions let CodeDeploy do things like:

- Accessing EC2 instances to deploy applications.
- Reading application artifacts from S3 buckets.
- Updating Auto Scaling groups.
- Write CloudWatch logs about what it's doing

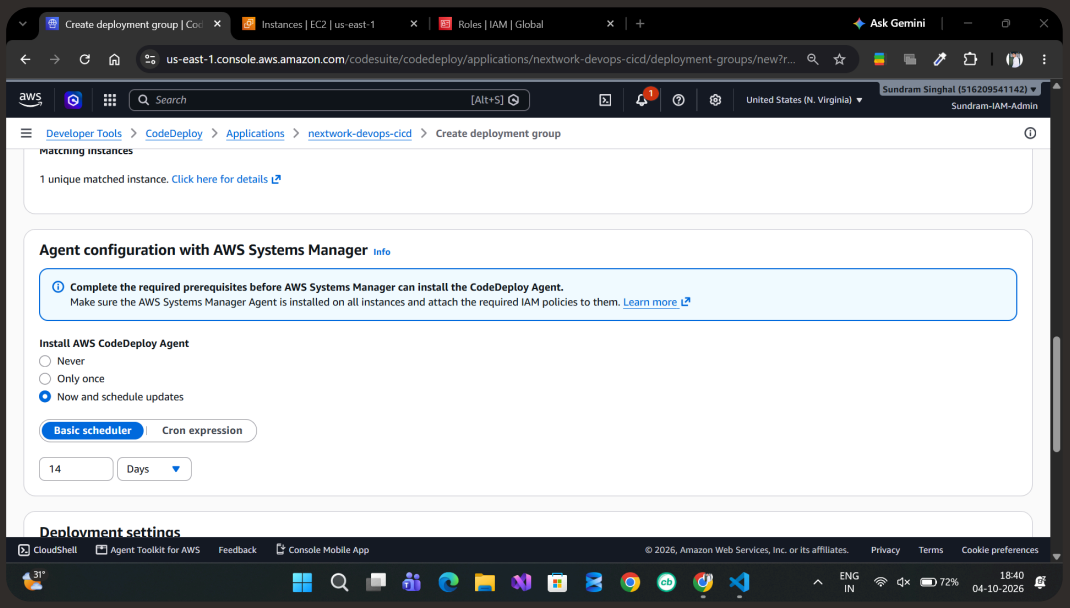
We are using Tags here so that CodeDeploy can identify the target EC2 instances for deployment. In our CloudFormation template, we tagged our EC2 instance with role: webserver. CodeDeploy will use this tag to find and deploy to the correct instance.



# Deployment configurations

Another key settings is the deployment configuration, which affects how quickly you'd like to roll out your application. I used CodeDeployDefault.AllAtOnce because It's the fastest option, but also the riskiest - if something goes wrong, all my instances could be affected at once.

The CodeDeploy Agent is a software that lets your EC2 instance communicate with CodeDeploy.
Whenever you initiate a deployment, it's the CodeDeploy Agent that receives the deployment instructions from CodeDeploy and carries them out on the EC2 instance.

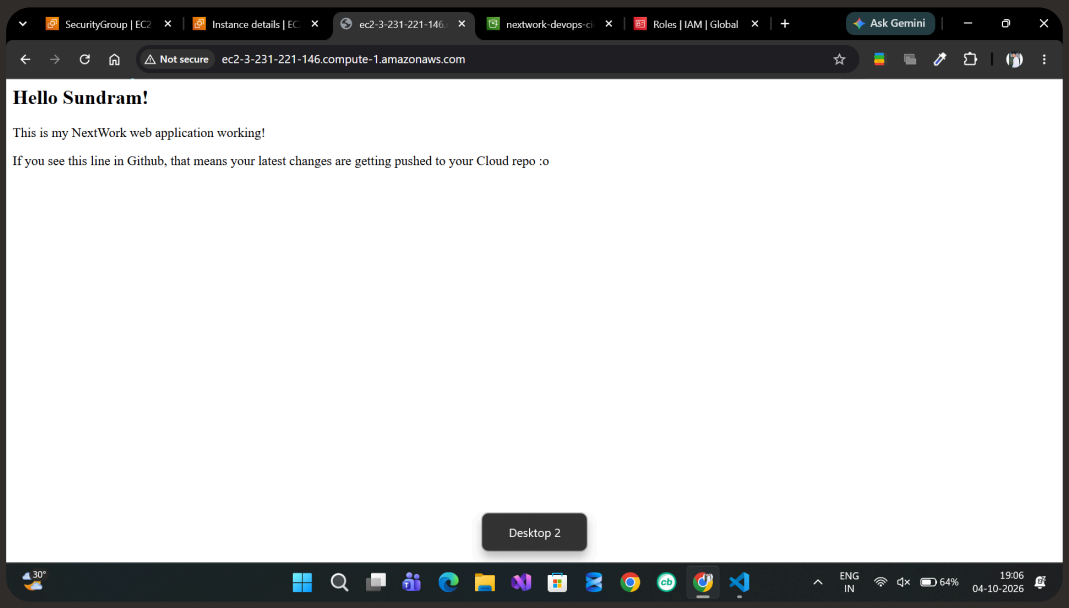


# Success!

A CodeDeploy deployment represents a single update to your application, with its own unique ID and history. When you create a deployment, you're telling CodeDeploy that Which version of the application to deploy, Where to deploy it and How to deploy it.

The revision location is the place where CodeDeploy looks to find your application's build artifacts. We're using the S3 bucket that's storing our WAR file, so CodeDeploy knows where to find the latest version of our web app it's deploying to the deployment EC2 instances!

To check that the deployment was a success in event sections all events lists get succeeded then I selected the Instance ID in the Deployment lifecycle events panel. This takes me to the deployment EC2 instance I launched with CloudFormation and then I clicked on Public IPv4 DNS and i took 2 minutes to open my delpoyed project.



# Disaster Recovery

In a project extension, I decided to make wrong chnges in my stop_server.sh file. The intentional error I created was i misspelled command that caused script to exit with non-zero status and the deployment got failed.

I also enabled rollbacks with this deployment, which means when w eenable rollback it head back our application to its previous state itself rather the spending so much time in finding errors in the deployment i.e rollbacks are super important because they help minimize downtime when things breaked.

When my deployment failed, the automatic rollback was not performed because it uses the latest code artifact To actually recover from the isuue while In production environments, true automated rollbacks are often implemented with more advanced tools. AWS CodePipeline can be configured to automatically roll back to the last successful deployment when a failure is detected, which eliminates the need for manual intervention.

